Anteater Chess User Manual

Version 1.0 Alpha Draft

By Jim Truong, Hamza Vaid, Shogo Stuck, Ben Choi,
Giovanna Dunker Estruquel
University of California, Irvine

Table of Contents

Glossary **3**
1. Computer Chess **5**
1.1. Usage Scenario 5
1.2. Goals 5
1.3. Features: 5
2. Installation **6**
2.1. System requirements 6
2.2. Setup and configuration 6
2.3. Uninstalling 7
3. Chess Program Functions and Features **7**
3.1. Game Interface 7
3.2. Player Moves 7
3.3. Computer Opponent 8
4. Back Matter **8**
Copyright 8
Error messages 8
Index 11

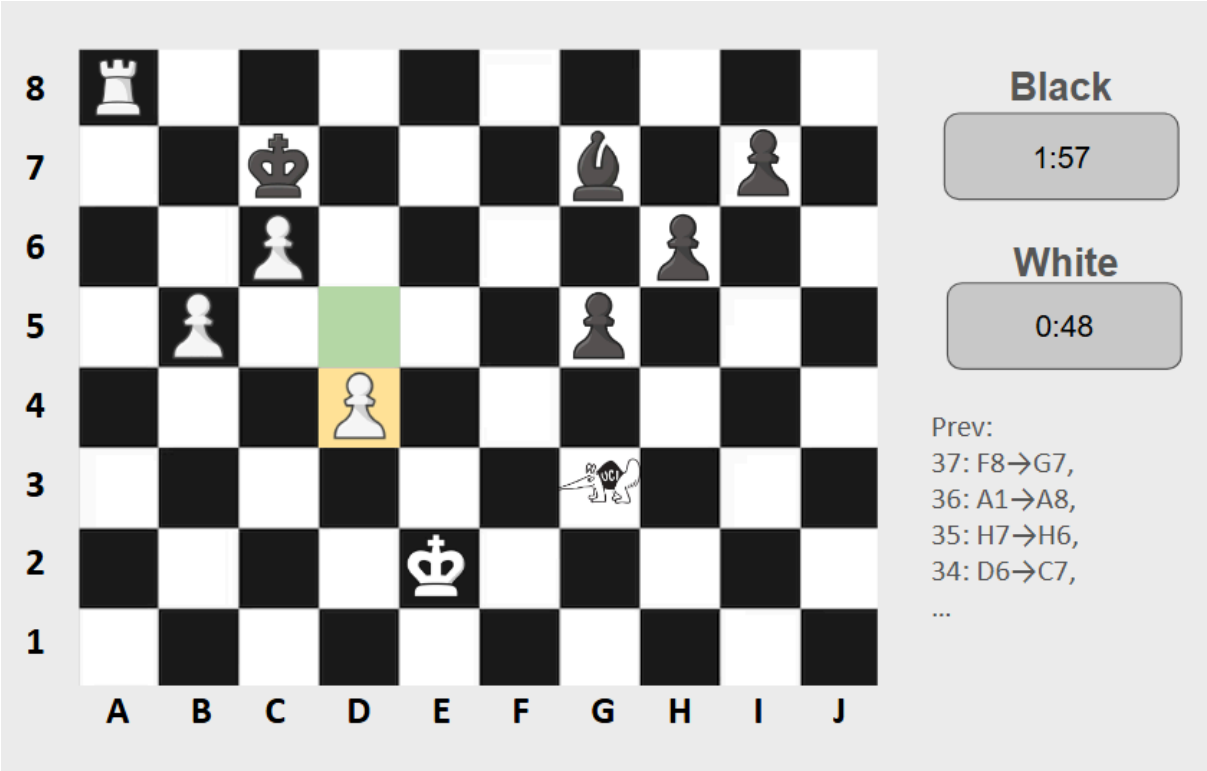
Glossary

- **Ant (Pawn)** – Can move forward one square (or two squares on its first move) and captures diagonally.
- **Anteater** – A special piece that can move one square in any direction (like a king), but can only capture ants (pawns). When capturing, the anteater can perform ant eating, meaning it can eat multiple adjacent ants in a single move (along the same row or column), ending on the position of the last ant eaten.
- **Ant Eating** – A special move where the anteater captures one or more adjacent ants in a sequence during a single turn. The anteater continues eating ants that are next to each other in the same row or column and finishes on the last square chosen by the player.
- **Bishop** – Can move any number of squares diagonally.
- **Black (Player)** – One of the two sides in the game. The black player typically moves second and controls the black-colored pieces.
- **Capture** – One piece attacks another, resulting in the removal of the attacked piece from the board.
- **Castling** – The king may move two pieces towards the rook, and the rook moves to the opposite side of the king, so long as:
    1. Neither the king nor the rook involved in the castle has moved in this game
    2. There are no pieces between the king and the rook involved in the castle
    3. The king is not in check at the time of the castle
    4. The king is not moving through any square in which it would be checked
- **Check** – When the king is under attack but has a legal move to escape.
- **Checkmate** – When the king cannot escape danger, and the game is over.
- **Computer (Player)** – The automated opponent controlled by the program, making moves based on implemented game logic and rules.
- **En Passant** – A special rule that allows the acting player to capture an ant (pawn) that just moved two squares forward on the square behind it using another pawn. The rule was added to make it more difficult to pass pawns on adjacent files.
- **File** – Synonymous with “columns,” which are labeled A - J

- **Human Player** – The user interacting with the program who inputs moves and controls one side, either white or black.
- **King** – The most important piece; moves one square in any direction and must not be checkmated.
- **Knight** – Moves in an L-shape and is the only piece that can jump over others.
- **Promotion** – When an ant reaches the last rank, it can be promoted to any piece (not a pawn).
- **Queen** – Can move any number of squares horizontally, vertically, or diagonally.
- **Rank** – Synonymous with "rows," which are labeled 1 - 8.
- **Rook** – Can move any number of squares horizontally and vertically.
- **Stalemate -** The acting player has no legal moves and is not in check, resulting in a draw
- **White (Player)** – One of the two sides in the game. The white player moves first and controls the white-colored pieces.

1. Computer Chess

1.1. Usage Scenario



Above is a possible screenshot of the finished GUI for anteater chess. Here, it's white to play and you can see the player has selected their D pawn (in yellow), highlighting the available squares (in green). There is also a timer for each player and a log showing previous moves at the bottom right corner of the screen.

1.2. Goals

- Allow a human player to play Anteater Chess against the computer.
- Follow official chess rules with the anteater extension.
- Provide a clear and interactive game experience.
- Ensure all moves are valid and legal.
- Provide a clear and intuitive interface.

1.3. Features:

The features of this application are categorized into two groups: required features, which are essential for the core functionality of the program, and optional

features, which are designed to enhance user experience and extend the program's capabilities.

Required Features:

- Interactive chess game (human vs computer).
- Display of chessboard and pieces.
- Move validation system based on chess rules.
- User selection of game side (white or black)
- Move logging (saved to file)
- Computer response times within a reasonable time (<1 minute)

Added Features (TBD):

- Human vs Human gameplay.
- Game timer.
- On-screen move logging.
- Highlighting legal moves.
- Check/Checkmate indicator.
- Drag and drop piece movement on GUI

2. Installation

2.1. System requirements

- Operating System: Windows / macOS / Linux
- Compiler: GCC (C language)
- Memory: TBD
- Storage: TBD

2.2. Setup and configuration

Currently, the program has not yet been developed, so a complete setup and configuration process is not available. However, the expected installation procedure is as follows:

1. Download and extract the program files.
2. Open terminal and navigate to project directory.

3. Compile the program using the command:

   `make`

4. Run the program using:

   `./chess`

(TBD: exact commands depending on final implementation)

2.3. Uninstalling

- Delete the project folder
- Remove compiled files if needed.

  (TBD) Depending on the file specifications and added features.

3. Chess Program Functions and Features

3.1. Game Interface

The game will use a 10x8 board. There are 8 ranks and 10 files. The bigger board in this variation of chess will feature 10 ants (pawns), 2 anteaters, 2 bishops, 2 rooks, 2 knights, a queen, and a king. This will be implemented as a two-dimensional array, where each element stores the piece currently occupying that square. A Piece structure will contain information about the piece type and color, and might also store whether the piece has been moved before to help with castling and en passant.

3.2. Player Moves

Player moves are conducted via chess notation. For example, a player input "F2 F4", the program will first check if there a piece on F2 then check what type of a piece. Then the program will run a check on all viable moves of that piece and check if the player's input is legal, if the correct player is moving the piece, and finally, the move is made on the board (*See error messages, p10*).

If the move played results in a check, there will be graphical feedback so the opposite player is aware they must escape the check.

If the move played results in a checkmate or a player's timer drops to zero, the game ends, notifying the user which player won and giving the user a prompt to exit the application or begin a new game.

3.3. Computer Opponent

The user will have the option to play a computer opponent. The computer will check all available moves and pick the best move to make. The first available algorithm will randomly select one of the legal moves to play, but future versions can improve this by using an evaluation function to score each board position. One possible approach is to assign point values to each piece, such as ants being worth 1 point, knights and bishops worth 3, etc. By comparing the total value of pieces remaining for each side, the computer opponent will be able to choose the best move to make. The evaluation function would also include factors that increase points for moves that place the opponent in check or protect important pieces. In later versions, this could be combined with a minmax algorithm so the computer not only looks at its own best move but also considers how the opponent might respond.

4. Back Matter

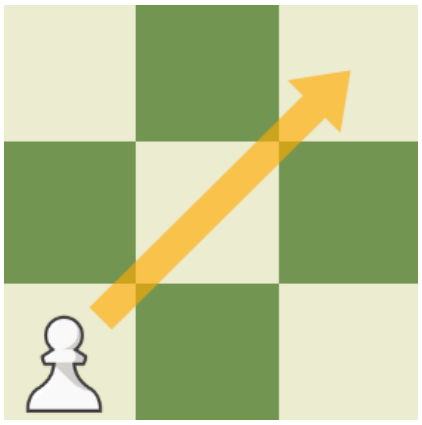
Copyright

© 2026 Anteater Chess Team (En Passant Police). All rights reserved.
This software was developed as part of the EECS 22L course at the University of California, Irvine and is intended for educational purposes only.

Error messages

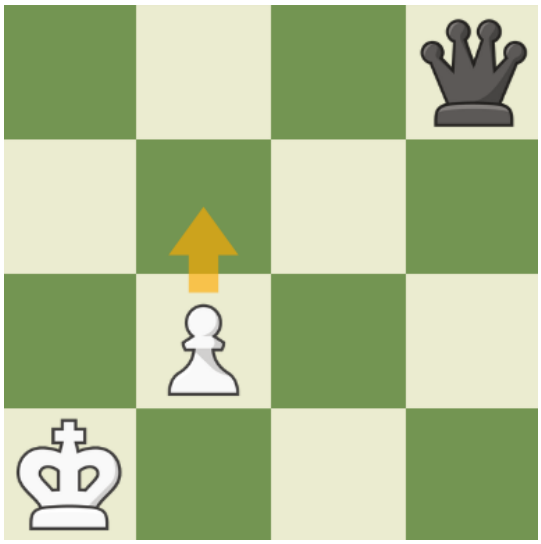
The Anteater Chess program provides feedback to the user when an invalid input or illegal action happens. Most of these will be rule violations pertaining to piece movement or king checks, but there are also other errors stemming from how the application runs. The following are error messages that will be implemented:

<u>Invalid Move</u> - A move that is inconsistent with how the pieces are allowed to act:

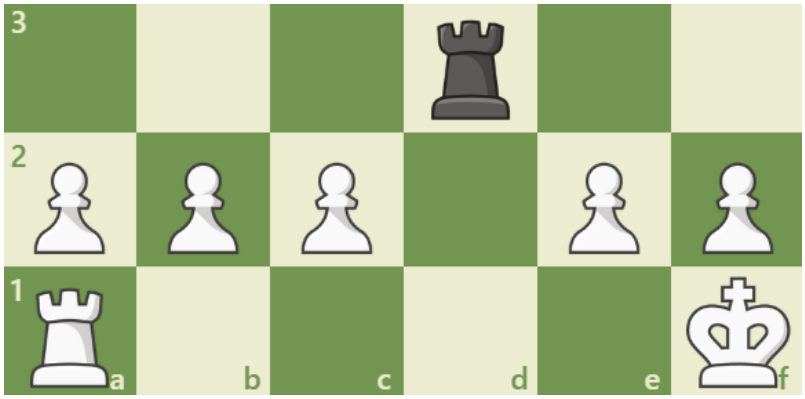


This move is not allowed because a pawn may only move one or two spaces forward, with the exception of captures which only allows one square of diagonal movement, not two.

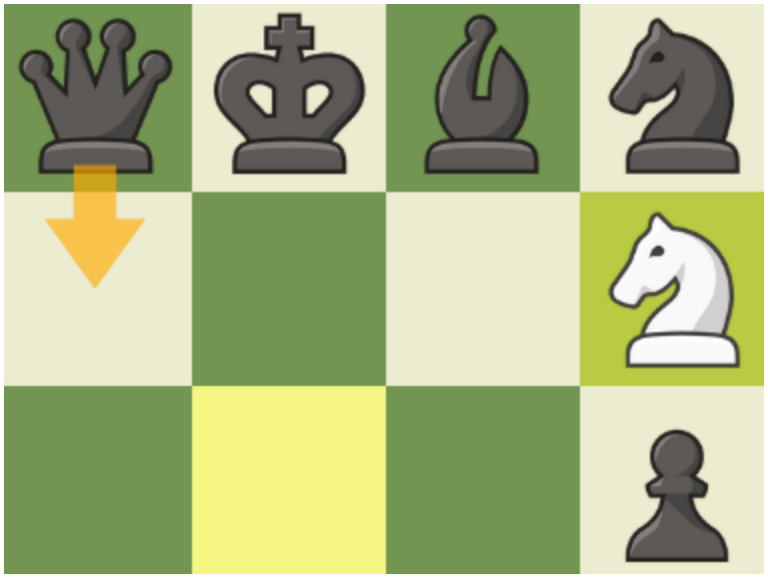
<u>Illegal Move</u> - A move consistent with piece movement, but not allowed due to checks



While ants are typically allowed to advance one square forward, white cannot play that move as it would place their king in check.



White would typically be allowed to castle here, but since the king would move through a square in which the rook checks the king, it is an illegal move.



In this position, the black king is in check, so moving the queen is illegal. This is because black must escape check by capturing the knight or moving the king.

Wrong Player Move - The same player can never play two moves in a row, so if a player attempts this, they will be met with a warning message

Move Out of Bounds - If a player specifies that they want to move a piece to or from a square outside the legal grid, lets say, F12 or Z3, they will be met with a warning message.

Missing File Error - If one of the game's files is missing, impeding the playability of the game, the application will provide an error message and the names of the missing files.

Index

A

Ant 3

Anteater 3

B

Bishop 3

Black 3, 6

C

Capture 3

Castling 3

Check 3

Checkmate 3

D

Download 6

E

En Passant 3

En Passant Police 8

F

File (Column) 3

File (Data) 6, 7, 10

I

Illegal Move 6

Invalid Move 6

K

King 4

Knight 4

O

Operating System 5

P

Pawn 3

Promotion 3

Q

Queen 4

R

Rank 4, 7

Rook 4

S

Stalemate 3

W

White 4, 6